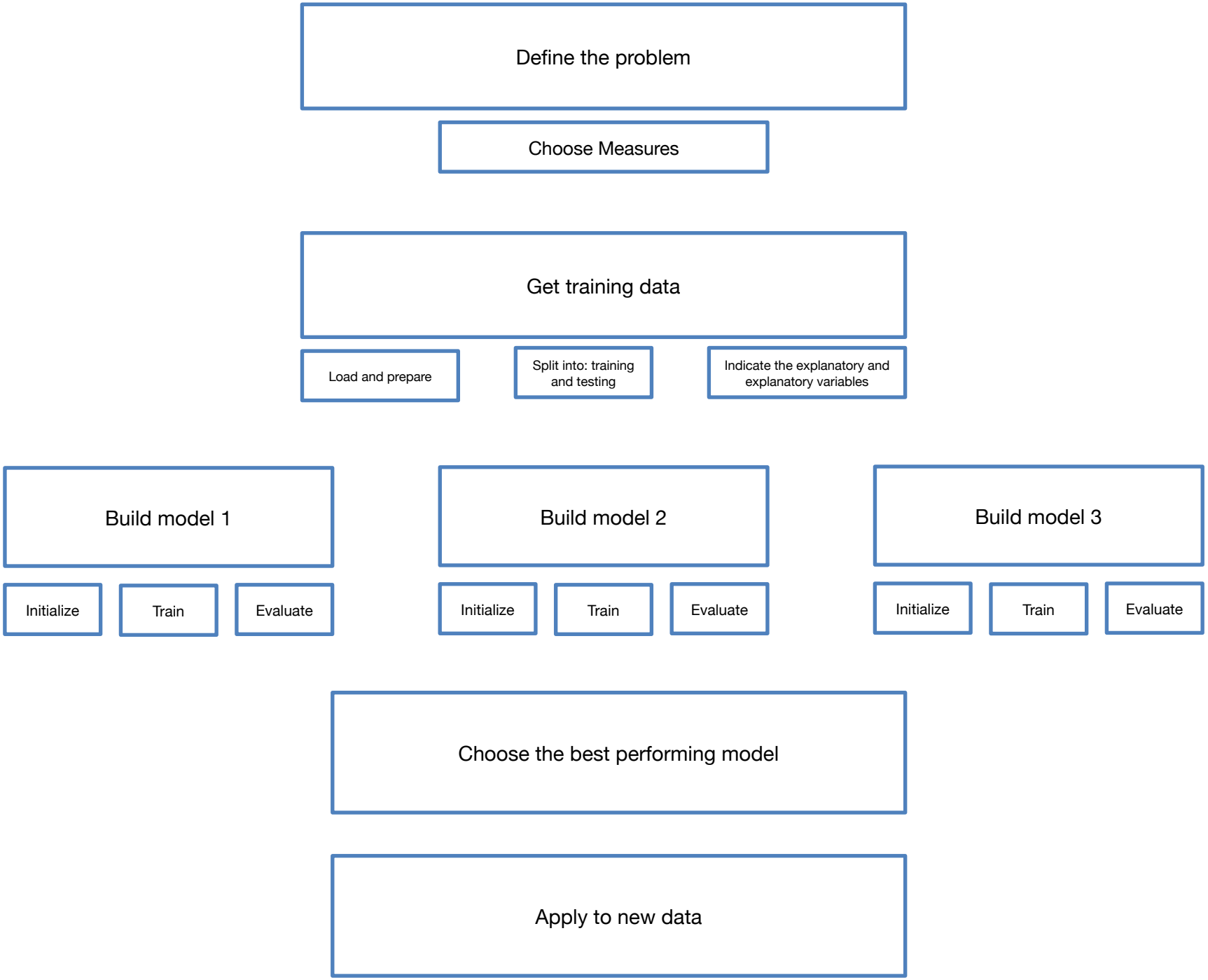
Machine Learning Process

Implementation in sklearn


Define the problem
Choose Measures
Get training data
Load and prepare
Split into: training and testing
Indicate the explanatory and explanatory variables
Build model 1
Initialize
Train
Evaluate
Build model 2
Initialize
Train
Evaluate
Build model 3
Initialize
Train
Evaluate
Choose the best performing model
Apply to new data

Get training data

- Load and prepare
- Split into: training and testing
- Indicate the explanatory and explanatory variables

Build the model

- Initialize
- Train
- Evaluate

Apply to new data

- Load new data
- Load model
- Apply the model to new data

Get training data

Load and prepare

Split into: training and testing

Indicate the explanatory and explanatory variables

[3]:
```
import pandas as pd
import numpy as np
df = pd.read_csv("Boston.csv", index_col=False)

df.head(3)
```

[3]:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
0	16.81180	0.0	18.1	0	0.700	5.277	98.1	1.4261	24	666	20.2	396.90	30.81	7.2
1	6.71772	0.0	18.1	0	0.713	6.749	92.6	2.3236	24	666	20.2	0.32	17.44	13.4
2	12.80230	0.0	18.1	0	0.740	5.854	96.6	1.8956	24	666	20.2	240.52	23.79	10.8

Get training data

Load and prepare

Split into: training and testing

Indicate the explanatory and explanatory variables

Train and test sets

First, let's randomly divide our collection into two parts:

1. training_set: on this data we will train models. Let's assume it will be 80% of the total set we have.
2. test_set: on this data we will test our models. This will be 20% of our entire dataset.

[5]:
```
from sklearn.model_selection import train_test_split

train, test = train_test_split(df, test_size=0.3)

#let's check shape
train.shape, test.shape
```

[5]:
```
((336, 14), (144, 14))
```

Get training data

Load and prepare

Split into: training and testing

Indicate the explanatory and explanatory variables

We will start by choosing the features on which we will train our model. Let's assume that these will be all features:
From: CRIM - per capita crime rate by town
To: LSTAT - lower status of the population

By convention, the relevant datasets are called:
X_train: the TRAINING set with explanatory characteristics
y_train: the TRAINING set with the feature we want to learn to predict.

55]:
```
features = [
    'crim',
    'zn',
    'indus',
    'chas',
    'nox',
    'rm',
    'age',
    'dis',
    'rad',
    'tax',
    'ptratio',
    'black',
    'lstat'
    ]
```

Build the model

Initialize

Train

Evaluate

Let's start by initializing the model. This will be our prototype, not yet trained...

```
from sklearn.linear_model import LinearRegression
model_initialized_1 = LinearRegression()
```

Build the model

Initialize

Train

Evaluate

Let us train our initial model on the training data ("fit" (fit) to this data)

```
model_trained_1 = model_initialized_1.fit(X_train,y_train)
```

Build the model

Initialize

Train

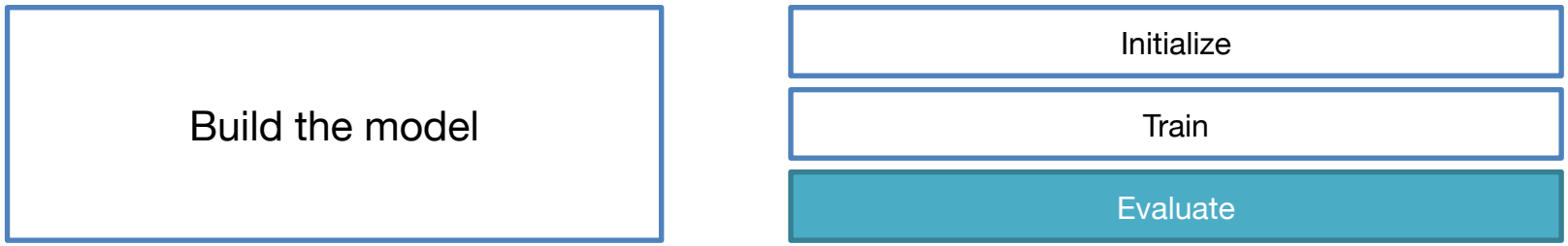
Evaluate

Now we will check how well our model predicts.

We first ask our model to make a prediction for the test set.
It looks as follows: as you can see, it does not contain a column with the predicted variable.

```
X_test.head()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat
355	0.04590	52.5	5.32	0	0.405	6.315	45.6	7.3172	6	293	16.6	396.90	7.60
296	0.11132	0.0	27.74	0	0.609	5.983	83.5	2.1099	4	711	20.1	396.90	13.35
170	8.79212	0.0	18.10	0	0.584	5.565	70.6	2.0635	24	666	20.2	3.65	17.16
335	0.54011	20.0	3.97	0	0.647	7.203	81.8	2.1121	5	264	13.0	392.80	9.59
340	20.71620	0.0	18.10	0	0.659	4.138	100.0	1.1781	24	666	20.2	370.22	23.34



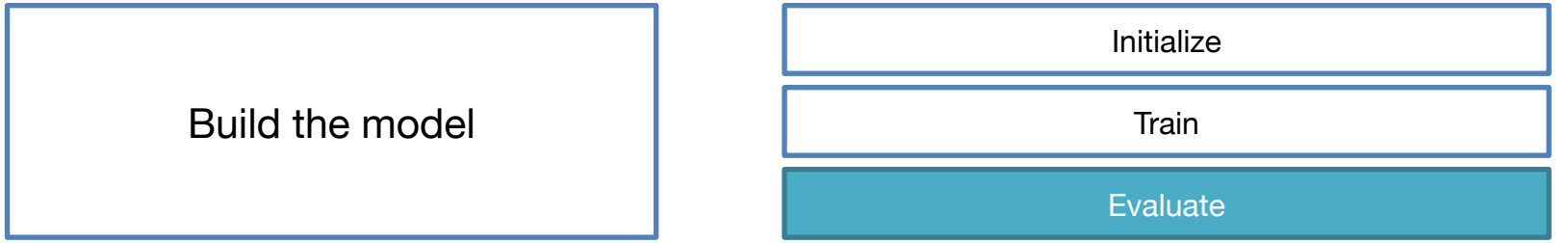
Now let's ask our model for its predictions

```
predictions = model_trained_1.predict(X_test)
```

Let's take a look at the forecast collection...

```
print(predictions)
print("\n")
print("The prediction set has %d values, and the test set has %d values." % (len(predictions), len(X_test)))
```

```
[26.66142233 16.00626568 14.68569786 35.67226521  9.30145121 20.91581138
 22.58437288 22.6530202  25.46258163 23.83845707 19.23219966 17.59192364
 14.58676062 10.53718797 26.41293239 28.53220094 30.30830165 15.44698169
 21.51926574 14.03928663 21.52074543 25.36871716 17.85162991 16.04924771
 19.29163479 22.05100887 15.20946922 17.13496327 18.87404149 23.01066609
```



Let us now compare the predictions of the model with the real data from the y_test set. We'll start with a pairwise comparison...

```
for i in range(len(predictions)):
    print(predictions[i], y_test.to_numpy()[i])
```

```
26.66142232558971 22.3
16.006265678127345 20.1
14.685697864649097 11.7
35.6722652058496 33.8
9.30145120804815 11.9
20.915811378145758 20.6
22.584372878767358 20.0
22.653020199326452 21.2
25.462581627222786 18.5
23.838457067249376 20.3
```

Build the model

Initialize

Train

Evaluate

Finally, let's calculate an aggregate measure of prediction accuracy: RMSE (Root Mean Square Error).

```
from sklearn.metrics import mean_squared_error
rmse_1 = np.sqrt(mean_squared_error(y_test, predictions))
print('RMSE = ', rmse_1)
```

```
RMSE =  4.48895290922776
```

How accurate is this? Let's calculate the spread of extreme price values.

```
print('Minimum value in the test set: %d' %(min(y_test)))
print('Maximum value in the test set: %d' %(max(y_test)))
```

```
Minimum value in the test set: 5
Maximum value in the test set: 50
```

Build the model

Initialize

Train

Evaluate

```
Let us now compare the performance of the different models.
```

```
models = [model_trained_1, model_trained_2, model_trained_3]
for i in range(3):
    print(models[i])
```

```
LinearRegression()
LinearRegression()
Ridge()
```

```
RMSE = [rmse_1, rmse_2, rmse_3]
for i in range(3):
    print('RMSE_',i+1, ' = ', RMSE[i])
```

```
RMSE_ 1  =  4.48895290922776
RMSE_ 2  =  7.923683635010311
RMSE_ 3  =  4.499969548158433
```

Exporting a model to a file

The best model is model_trained_1. So let's save it to a file so that we can use it later in practice.

```
import pickle

pickle.dump(model_trained_1, open('our_regressor.sav', 'wb'))
```

Apply the model to new data

Load new data

Load model

Apply the model to new data

Let's start by loading a file with the characteristics of locations for which we want to calculate prices of apartments

```
import pandas as pd

unknowns = pd.read_csv("Boston_unknowns.csv")

unknowns.head()
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat
0	0.05083	0.0	5.19	0	0.515	6.316	38.1	6.4584	5	224	20.2	389.71	5.68
1	0.14030	22.0	5.86	0	0.431	6.487	13.0	7.3967	7	330	19.1	396.28	5.90
2	0.24522	0.0	9.90	0	0.544	5.782	71.7	4.0317	4	304	18.4	396.90	15.94
3	0.02055	85.0	0.74	0	0.410	6.383	35.7	9.1876	2	313	17.3	396.90	5.77
4	0.01381	80.0	0.46	0	0.422	7.875	32.0	5.6484	4	255	14.4	394.23	2.97

Apply the model to new data

Load new data

Load model

Apply the model to new data

```
import pickle
our_regressor = pickle.load(open('our_regressor.sav', 'rb'))
```

Apply the model to new data

Load new data

Load model

Apply the model to new data

```
predictions = our_regressor.predict(unknowns)
```

```
print(predictions)
```

```
[21.35609066 23.28723301 18.60243767 24.00528218 41.02580616 18.16502006
 18.70215048 35.60643169 19.09193562 31.52991567 29.21996001 32.37900035
 19.22335814 20.59099419 35.78599158 24.60835985 17.32963563 18.68210327
 35.9715064  21.47758029 22.27697944 31.56075936 13.71638488 20.96631641
 32.36680778 15.76743562]
```

Apply the model to new data

Load new data

Load model

Apply the model to new data

```
# as you can see, the predictions are in the form of ndarray. Let's turn them into a dataframe
predictions_df = pd.DataFrame(predictions, columns = ['predictions'])
```

```
# ... and merge it with the location features
data_with_predictions = pd.concat([unknowns, predictions_df], axis = 1)
```

```
data_with_predictions
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	predictions
0	0.05083	0.0	5.19	0	0.515	6.316	38.1	6.4584	5	224	20.2	389.71	5.68	21.356091
1	0.14030	22.0	5.86	0	0.431	6.487	13.0	7.3967	7	330	19.1	396.28	5.90	23.287233
2	0.24522	0.0	9.90	0	0.544	5.782	71.7	4.0317	4	304	18.4	396.90	15.94	18.602438
3	0.02055	85.0	0.74	0	0.410	6.383	35.7	9.1876	2	313	17.3	396.90	5.77	24.005282
4	0.01381	80.0	0.46	0	0.422	7.875	32.0	5.6484	4	255	14.4	394.23	2.97	41.025806
5	7.02259	0.0	18.10	0	0.718	6.006	95.3	1.8746	24	666	20.2	319.98	15.70	18.165020
6	9.59571	0.0	18.10	0	0.693	6.404	100.0	1.6390	24	666	20.2	376.11	20.31	18.702150
7	0.10469	40.0	6.41	1	0.447	7.267	49.0	4.7872	4	254	17.6	389.25	6.05	35.606432
8	0.24103	0.0	7.38	0	0.493	6.083	43.7	5.4159	5	287	19.6	396.90	12.79	19.091936